Relatório de atividades de implementação de algoritmos para resolução do problema das 8 rainhas

Alunos: Marcos Abreu e Ricardo Colli

Disciplina: Inteligência Artificial

Sumário

1 Proposta do trabalho....2
2 Linguagem de programação utilizada....2
3 Descrição do código fonte....2
3.1 Bibliotecas utilizadas e observações....2
3.1.1 Bibliotecas....2
3.1.2 Funcionamento do problema das 8 rainhas....2
3.1.3 Comportamento das rainhas....2
3.2 Geração de estado aleatório inicial....2
3.3 Contador de conflitos....3
3.4 Gerador de vizinhos....3
3.5 Subida de encosta....4
3.6 Subida de encosta com reinício....5
3.7 Têmpera simulada....6
3.8 Interface....7
3. 9 Desempenho dos algoritmos....7
3.9.1 Observações dos testes....8
3.9.1.1 Resultados da Subida de encosta....8
3.9.1.2 Resultados da Subida de encosta com Reinício....8
3.9.1.3 Tempêra Simulada....9
3.9.2 Conclusões sobre os algoritmos....9
3.10 Acesso do código....11
4 Referências....11

1 Proposta do trabalho

Implementar algoritmos para encontrar a melhor solução dado ao estado inicial do jogo das 8 rainhas.

2 Linguagem de programação utilizada

Python

3 Descrição do código fonte

3.1 Bibliotecas utilizadas e observações

3.1.1 Bibliotecas

Para o desempenho dos algoritmos foram utilizadas duas bibliotecas, sendo elas 'import random' e 'import math' tendo por objetivo respectivamente uma função para gerar números aleatórios e também para conseguir manipular cálculos matemáticos mais complexos.

3.1.2 Funcionamento do problema das 8 rainhas

O problema das 8 rainhas é um clássico quebra-cabeça de lógica e computação que consiste em posicionar oito rainhas em um tabuleiro de xadrez padrão (8x8) de forma que nenhuma rainha ataque outra

3.1.3 Comportamento das rainhas

As rainha dentro do código tem colunas fixas e só se movimentam entre as linhas, armazenando a posição de suas colunas pelo valor do índice de uma lista e a sua linha pelo valor presente no seu índice.

3.2 Geração de estado aleatório inicial

Para gerar estados iniciais foi implementada uma função que gera números aleatórios entre 0 até 7 (possível posição de certa rainha na sua determinada linha).

3.3 Contador de conflitos

No presente código teve a implementação de um contador de conflitos, onde ele analise se uma determinada rainha tem algum conflito com alguma outra tanto na mesma linha quanto na diagonal.

Figura 1: Função para contar conflitos

```python
def conflitos(tabuleiro):
    conflitos = 0
    for i in range(N):
        for j in range(i + 1, N):
            if tabuleiro[i] == tabuleiro[j]:
                conflitos += 1
            if abs(tabuleiro[i] - tabuleiro[j]) == abs(i - j):
                conflitos += 1
    return conflitos
```

Fonte: Autor

3.4 Gerador de vizinhos

A função geradora de vizinhos tem como objetivo produzir todos os estados possíveis a partir do estado atual, movendo cada rainha dentro de sua respectiva coluna para todas as outras posições de linha. Para cada coluna do tabuleiro, a função testa todas as possíveis linhas (de 0 a 7), exceto a posição atual da rainha, gerando assim novos estados candidatos. Cada novo estado é criado como uma cópia do estado original, alterando apenas a posição de uma única rainha por vez. Essa abordagem garante que o espaço de busca seja explorado localmente, permitindo que os algoritmos comparem estados vizinhos e escolham aquele com menor número de conflitos.

O número total de vizinhos gerados em cada iteração é dado por N × (N - 1), onde N é o número de rainhas. No caso do problema das 8 rainhas, isso resulta em até 56 vizinhos por estado.

Figura 2: Função para geração de novos vizinhos

```python
def gerar_vizinhos(tabuleiro):
    vizinhos = []
    for col in range(N):
        for linha in range(N):
            if linha != tabuleiro[col]:
                novo = tabuleiro.copy()
                novo[col] = linha
                vizinhos.append(novo)
    return vizinhos
```

Fonte: Autor

3.5 Subida de encosta

O algoritmo de subida de encosta (*Hill Climbing*) é uma técnica de busca local que foi implementada em nosso código e funciona de forma que ele começa com um estado inicial e a partir disso irá gerar os possíveis vizinhos e escolher o melhor vizinho para ser o próximo passo, o melhor vizinho é definido pelo próximo estado com o menor número de conflitos e assim seguindo em frente enquanto tiver estados com menos conflitos ou quando mesmo com um estado com mais de 0 conflitos e não existir mais novos estados possíveis para serem testados (ficando preso em um mínimo local). Embora seja simples e eficiente, o algoritmo não garante encontrar a solução ótima, pois pode ficar preso em mínimos locais.

Figura 3: Algoritmo de subida de encosta

```python
def subida_encosta(estado):
    atual = estado.copy()
    estados_testados = 0
    caminho = [atual.copy()]

    while True:
        vizinhos = gerar_vizinhos(atual)
        melhor = atual
        melhor_conflito = conflitos(atual)

        for v in vizinhos:
            estados_testados += 1
            c = conflitos(v)
            if c < melhor_conflito:
                melhor = v
                melhor_conflito = c

        if melhor_conflito >= conflitos(atual):
            return atual, estados_testados, caminho

        atual = melhor
        caminho.append(atual.copy())
```

Fonte: Autor

3.6 Subida de encosta com reinício

A subida de encosta com reinício segue o mesmo princípio da subida de encosta tradicional, porém, quando o algoritmo fica preso em um mínimo local, um novo estado inicial aleatório é gerado e o processo é reiniciado.

Esse procedimento é repetido diversas vezes até que seja encontrada uma solução com zero conflitos ou até que um número máximo de reinícios seja atingido.

Sendo assim o algoritmo aumenta a probabilidade de encontrar o estado ideal porém podendo ter um grande custo de tempo tendo em vista que poderá ficar com um grande acúmulo de reinícios (Limitado a 100 no código).

Figura 4: Subida de encosta com reinício

```
def subida_encosta_reinicio(estado_inicial=None, max_reinicios=100):

    total_reinicios = 0
    total_estados = 0

    melhor_caminho = []

    while total_reinicios < max_reinicios:

        if total_reinicios == 0 and estado_inicial is not None:
            estado = estado_inicial.copy()
        else:
            estado = estado_aleatorio()

        resultado, estados, caminho = subida_encosta(estado)
        total_estados += estados

        if conflitos(resultado) == 0:
            return resultado, total_reinicios, total_estados, caminho

        melhor_caminho = caminho
        total_reinicios += 1

    return resultado, total_reinicios, total_estados, melhor_caminho
```

Fonte: Autor

3.7 Têmpera simulada

O método de otimização heurística simulado templo ou Simulated Annealing (AS), é um algoritmo utilizado para problemas de otimização global, que visa encontrar uma boa aproximação do ponto ótimo geral, ou seja, o ponto mínimo global para todo o espaço de solução, conseguindo várias modificações que pioram ou temporariamente fugir das soluções ótimas locais [1].

O algoritmo inicia com um estado inicial e uma temperatura elevada (1000). A cada iteração, um novo estado vizinho é gerado aleatoriamente, modificando a posição de uma rainha. Em seguida, é calculada a variação de conflitos entre o novo estado e o estado atual. Caso o novo estado seja melhor, ele é sempre aceito. Caso seja pior, ele ainda pode ser aceito com uma probabilidade calculada pela função exponencial, que depende da diferença de conflitos e da temperatura atual.

Ao longo da execução, a temperatura é gradualmente reduzida por meio de um fator de resfriamento, diminuindo a probabilidade de aceitar soluções piores. Dessa forma, o algoritmo passa de uma fase de exploração para uma fase de refinamento.No código implementado, foi utilizada uma temperatura inicial elevada, um fator de resfriamento de 0.95 e um limite máximo de iterações. Essa configuração permite um bom equilíbrio entre exploração do espaço de busca e convergência para soluções com menor número de conflitos. A principal vantagem da têmpera simulada é sua capacidade de escapar de mínimos locais, aumentando a probabilidade de encontrar uma solução global.

3.8 Interface

Para a demonstração do caminho percorrido foi utilizado uma interface parcialmente gerada com Inteligência Artificial usando o Tkinter onde o usuário define qual algoritmo irá utilizar e então poderá gerar um estado inicial e clicar no botão para resolver. Após isso, logo abaixo do tabuleiro irão aparecer os resultados obtidos.

Figura 5: Interface

Fonte: Autor

3. 9 Desempenho dos algoritmos

Para a coleta de desempenho foram coletadas 5 execuções de cada algoritmo para o comparativo.

Figura 6: Testes dos algoritmos

Estado inicial: [7, 3, 4, 7, 2, 2, 7, 6]
Algoritmo: Subida de Encosta
Conflitos finais: 2
Estados testados: 280
Movimentos: 4
Tempo: 0.0014s

Estado inicial: [5, 7, 7, 0, 1, 6, 0, 6]
Algoritmo: Subida de Encosta
Conflitos finais: 1
Estados testados: 280
Movimentos: 4
Tempo: 0.0013s

Estado inicial: [0, 0, 3, 7, 3, 0, 4, 0]
Algoritmo: Subida de Encosta
Conflitos finais: 0
Estados testados: 336
Movimentos: 5
Tempo: 0.0017s

Estado inicial: [1, 6, 5, 0, 3, 5, 5, 3]
Algoritmo: Subida de Encosta
Conflitos finais: 2
Estados testados: 224
Movimentos: 3
Tempo: 0.0012s

Estado inicial: [7, 6, 3, 2, 3, 5, 3, 6]
Algoritmo: Subida de Encosta
Conflitos finais: 1
Estados testados: 224
Movimentos: 3
Tempo: 0.0011s

Estado inicial: [4, 1, 1, 3, 7, 5, 0, 7]
Algoritmo: Subida com Reinício
Conflitos finais: 0
Estados testados: 2016
Movimentos (total): 35
Reinícios: 7
Tempo: 0.0101s

Estado inicial: [4, 1, 0, 5, 7, 1, 2, 3]
Algoritmo: Subida com Reinício
Conflitos finais: 0
Estados testados: 616
Movimentos (total): 10
Reinícios: 2
Tempo: 0.0031s

Estado inicial: [2, 6, 0, 3, 6, 1, 1, 7]
Algoritmo: Subida com Reinício
Conflitos finais: 0
Estados testados: 896
Movimentos (total): 15
Reinícios: 2
Tempo: 0.0043s

Estado inicial: [6, 6, 2, 6, 3, 4, 1, 4]
Algoritmo: Subida com Reinício
Conflitos finais: 0
Estados testados: 448
Movimentos (total): 7
Reinícios: 1
Tempo: 0.0022s

Estado inicial: [5, 4, 0, 2, 6, 1, 5, 1]
Algoritmo: Subida com Reinício
Conflitos finais: 0
Estados testados: 5824
Movimentos (total): 103
Reinícios: 25
Tempo: 0.0277s

Estado inicial: [1, 0, 0, 5, 1, 1, 3, 3]
Algoritmo: Têmpera Simulada
Conflitos finais: 1
Estados testados: 270
Movimentos (total): 157
Tempo: 0.0066s

Estado inicial: [5, 5, 0, 4, 3, 0, 4, 2]
Algoritmo: Têmpera Simulada
Conflitos finais: 1
Estados testados: 270
Movimentos (total): 182
Tempo: 0.0067s

Estado inicial: [6, 7, 4, 5, 6, 1, 0, 4]
Algoritmo: Têmpera Simulada
Conflitos finais: 0
Estados testados: 225
Movimentos (total): 145
Tempo: 0.0055s

Estado inicial: [0, 6, 3, 0, 1, 2, 1, 5]
Algoritmo: Têmpera Simulada
Conflitos finais: 1
Estados testados: 270
Movimentos (total): 169
Tempo: 0.0064s

Estado inicial: [2, 6, 5, 4, 0, 4, 4, 0]
Algoritmo: Têmpera Simulada
Conflitos finais: 0
Estados testados: 265
Movimentos (total): 153
Tempo: 0.0063s

Fonte: Autor

Para a têmpera após esses testes foi alterada a temperatura inicial de 1000 para 2000 e taxa de resfriamento de 0.95 para 0.98, obtendo mais vezes estados final com 0 conflitos.

3.9.1 Observações dos testes

3.9.1.1 Resultados da Subida de encosta

Observação dos testes:

- Conflitos finais: 0, 1 ou 2
- Estados testados: ~224 a 336
- Movimentos: 3 a 5
- Tempo: muito baixo (~0.0013s)

A subida de encosta apresentou excelente desempenho em tempo e número de estados testados, porém não garantiu a obtenção de soluções ótimas em todas as execuções.

3.9.1.2 Resultados da Subida de encosta com Reinício

Observação:

- Conflitos finais: sempre 0
- Estados testados: bem maiores (448 até 5824)
- Movimentos totais: 7 até 103
- Reinícios: 1 a 7
- Tempo: maior, mas ainda rápido

A estratégia de reinício aumentou significativamente a capacidade do algoritmo em encontrar soluções ótimas, eliminando o problema dos mínimos locais. No entanto, aumentou o custo computacional, podendo ter um maior número de estados testados e movimentos totais.

3.9.1.3 Tempêra Simulada

Antes:

- Conflitos finais: 0 ou 1
- Instável

Depois (com seus parâmetros novos):

- Temperatura inicial: 2000
- Resfriamento: 0.98
- Limite maior

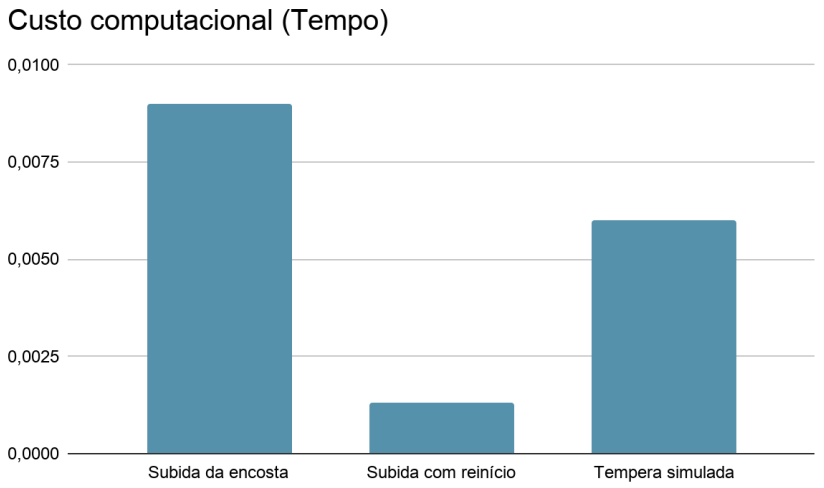
Resultado:

- Muito mais casos com conflito 0
- Mais consistente

A têmpera simulada apresentou comportamento probabilístico, podendo encontrar soluções ótimas ou quase ótimas. Após o ajuste dos parâmetros da temperatura inicial e a redução da taxa de resfriamento, o algoritmo passou a explorar melhor o espaço de busca, aumentando significativamente a frequência de soluções com zero conflitos.

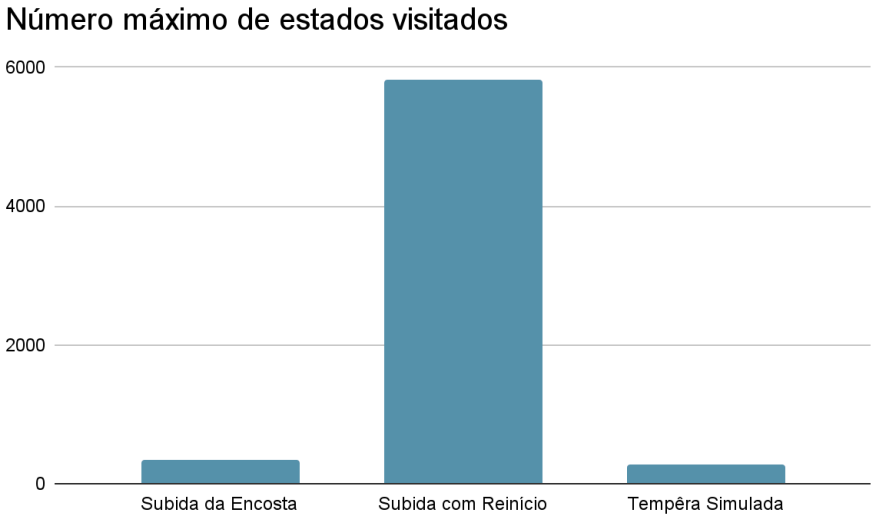
3.9.2 Conclusões sobre os algoritmos

Gráfico 1: Tempo gasto médio em segundos para solução final



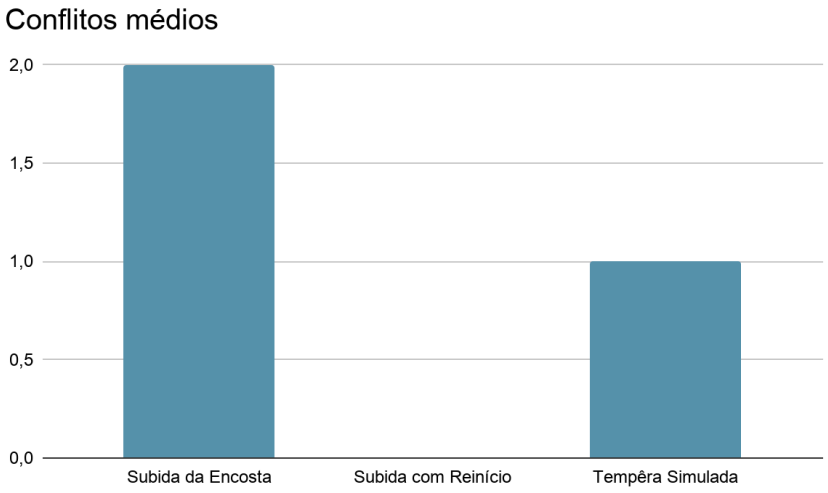
Fonte: Autor

Gráfico 2: Estados visitados



Fonte: Autor

Gráfico 3: Conflitos finais médio



Fonte: Autor

Com base nos experimentos realizados, conclui-se que a subida de encosta é o algoritmo mais eficiente em termos de tempo e número de movimentos, porém apresenta baixa confiabilidade na obtenção de soluções. A subida de encosta com reinício mostrou-se altamente eficaz, encontrando soluções com zero conflitos em todas as execuções, embora com maior custo computacional. A têmpera simulada, especialmente após o ajuste dos parâmetros, apresentou um equilíbrio entre eficiência e eficácia, conseguindo encontrar soluções ótimas na maioria dos casos com custo computacional moderado.

Dessa forma, a têmpera simulada (com os parâmetros ajustados) se destaca como a melhor escolha geral para o problema das 8 rainhas, por equilibrar qualidade da solução e desempenho, enquanto a subida com reinício é mais indicada quando a prioridade é garantir a solução ótima.

Tabela 1: Resumo dos algoritmos

Algoritmo	**Eficiência**	**Qualidade**
Encosta	Alta	Média
Reinício	Média	Alta
Têmpera	Equilibrado	Boa

Fonte: Autor

3.10 Acesso do código

Código está vinculado ao GitHub: https://github.com/RicardoColli/8puzzle/tree/main/rainhas

4 Referências

[1] DE OLIVEIRA, S.A.; DE ALMEIDA, C.R.T.; MONTICELLI, A. Times assíncronos aplicados a métodos heurísticos construtivos de planejamento da expansão da transmissão. In: CONGRESSO BRASILEIRO DE AUTOMATICA - CBA, 12., 1998, Uberlândia. Proceedins... Uberlândia: SBA/UFU, v.III, 1998. p.1029- 1034